Thesis Proposal

An Adaptive Wave Director for a Maze Tower Defense

A contextual bandit that answers the weaknesses of the player's build, measured by whether players rebuild

[Student name]
ID: [student ID]

Advisor: [advisor name]

Computer Technology (CT)
Department of Computer Engineering / DDCT
King Mongkut's University of Technology Thonburi (KMUTT)

October 2026

Table of Contents

1. Introduction 6
1.1 Background and Problem Statement 6
1.2 Goals and Objectives 6
1.3 Hypotheses 7
1.4 Scope 7
In scope 7
Out of scope 8
Limitations and requirements 8
1.5 How to Read This Document 8
2. Literature Review 9
2.1 Adaptive Game AI 9
2.2 Related Works 9
2.3 Algorithms and Techniques 10
2.3.1 Finding paths with a flow field 10
2.3.2 A repeatable game 11
2.3.3 Learning which attack works: contextual bandits and Thompson sampling 11
2.3.4 Measuring how much players rebuild 12
2.4 Approaches Considered and Rejected 12
2.5 Tools for Development 13
3. Deliverable Details 14
3.1 Project Overview 14
3.2 System Architecture 16
3.3 Game Core 17
3.4 How a Wave is Planned 18
3.5 Adaptive Director 19
3.5.1 Base analysis 19
3.5.2 Attack types and the shared enemy price list 20
3.5.3 The three steps of a decision 20
3.5.4 Reward 20
3.6 Saved Data and Replays 21
3.7 Current Status 21
4. Evaluation Methodology 23
4.1 Checkpoints 23
4.2 Testing With Computer Players 23
4.3 Study With Real Players 23
4.4 What Could Make the Results Wrong 24
5. Plan of Action 25
5.1 Tasks 25
5.2 Schedule 26

5.3 Monitoring and Evaluation 27
5.4 Risks 27
6. References 28
Appendix A. Glossary of Technical Terms 30

List of Figures

Figure 1 The flow-field view on a 48 × 48 map made from a seed
Figure 2 Start of a session
Figure 3 Building between waves
Figure 4 The director at work (development version)
Figure 5 A wave in progress
Figure 6 The parts of the program and how they depend on each other
Figure 7 What happens in one game step, and the two files a session leaves behind
Figure 8 The steps of the director
Figure 9 After a wave

List of Tables

Table 1 Hypotheses and how each is tested
Table 2 Comparison with related adaptive systems
Table 3 Rejected approaches and the reason
Table 4 Development tools
Table 5 Size of the code at the time of writing
Table 6 Fixed rules of the project
Table 7 The five measurements of the player’s base (first definitions; fixed before the trial study)
Table 8 What is built and what is planned
Table 9 Checkpoints
Table 10 Work packages
Table 11 Semester 1 schedule
Table 12 Break and semester 2 schedule
Table 13 Input, process, output and benefit
Table 14 Main risks and how to handle them
Table 15 Glossary of technical terms

Abstract

In a maze tower defense game, the player builds walls to make the enemies' path longer and places towers along that path to shoot them. A good player usually finds one strong layout within a few waves and then keeps using it, because the waves are fixed and never react to what the player built. This project proposes an **adaptive wave director**: a part of the game that looks at the player's base before every wave and chooses what kind of attack to send next (which enemy types, from which sides, aimed at which weak point). The total strength of every wave stays the same as in the normal, non-adaptive game.

The director decides in three steps, always in the same order. First, fixed rules remove choices that are not allowed or not fair. Second, hand-written scoring rules give each kind of attack a score based on five measurements of the player's base. Third, a small learning part adds a correction of at most ±1 to each score. This learning part is a *contextual bandit* that uses *Thompson sampling* (both are explained in the glossary). The learner is rewarded when the player **changes their base** after a wave, not when the player loses.

The game rules run in a separate C# program that gives exactly the same result every time it is given the same input. It runs in fixed time steps of 0.02 seconds, and Unity is used only to show the game and read the mouse and keyboard. Because of this, the same rules can run without Unity for automatic tests, replays and about a million practice games. Every session saves a replay and one line of data per wave. The main measurement, how spread out the player's building is across the map, is calculated from that data afterwards. The idea is tested first with computer-controlled players (checkpoints G1–G4) and then with about 24 real players, who each play both versions of the game. The claim being tested is not that the director wins more often, but that players **rebuild more**.

1. Introduction

1.1 Background and Problem Statement

Tower defense is a type of strategy game in which the player places towers to stop waves of enemies from reaching a target. In a *maze* tower defense the player also builds walls, so the path itself becomes part of the plan: a longer path keeps enemies under fire for longer. The genre rewards planning, but it has a well-known weakness. Once the player finds a layout that works, the fixed waves keep testing the same thing, and the best answer is to repeat that layout. From then on the player stops making real decisions about the base, and the game turns into routine repair work.

Games have adjusted themselves to the player for a long time. Dynamic difficulty adjustment (DDA) makes enemies stronger or weaker depending on how well the player is doing (Hunicke, 2005; Yannakakis & Togelius, 2018), and the "AI director" in *Left 4 Dead* controls how many enemies appear based on an estimate of the players' stress (Booth, 2009). These systems mostly change *how hard* the game is or *how fast* it feels. They do not try to change *how* the player plays. A harder wave that the same layout can still handle does not change the player's behaviour. Also, a director that simply makes the game harder makes a fair comparison impossible, because "it was harder" could explain any difference.

This project aims at the player's behaviour directly. The director finds the weak points of the current base (an open gap, an unprotected outpost, all towers in one place, only one type of damage, little defence against flying enemies) and sends a wave that attacks one of them. It must spend exactly the same amount of enemy strength as the normal wave it replaces. If the player answers by changing the base, the director has done its job; if the player ignores it and loses, it has not. The research question is therefore:

> **Research question.** If a wave director changes the *kind* of each attack to match the player's base, while keeping the difficulty the same, do players change their defences more often than when the waves simply get stronger in a fixed way?

Two practical problems must be solved to answer this question properly. First, a learning director has very little data: a session lasts about 25 waves, so the learner makes only about 25 decisions per player. Second, the comparison must be fair: both versions of the game must use the same map, the same ore deposits and enemy entry points, and the same total enemy strength at every wave. Most of the technical work in this proposal exists to solve these two problems.

1.2 Goals and Objectives

The project will deliver a playable game, an adaptive director inside it, a test of the director with computer-controlled players, and a study with real players. Each item below can be checked against the finished product.

1. **A repeatable game core.** The same map, random seed and player input always give exactly the same game, both inside Unity and without it. This is checked by saving a fingerprint (hash) of the whole game state at every step (rule I1).
2. **A playable maze tower defense** built on that core: walls that enemies can dig through instead of walls that block, towers, an ore economy, several kinds of enemy movement, and a normal version where waves simply get stronger. The game design is described in the separate Game Design Document (GDD).

3. **A base analyser** that turns the player's base into five numbers (`maze_length`, `breach_vulnerability`, `chokepoint_reliance`, `tower_concentration`, `damage_type_mix`).
4. **A set of attack types and one shared price list for enemies**, so that every attack type costs exactly the same enemy strength at the same wave number (rule I2).
5. **A three-step director**: fixed rules that remove choices, hand-written scores, and a small learned correction. The learner keeps a score table per situation, borrows from similar past situations when a table is nearly empty, and slowly forgets old results.
6. **Data and replays:** one line of data per wave and one replay file per session, from which the main measurement can be calculated again exactly.
7. **A test bench** in which computer-controlled players play many games, to check the director (checkpoints G1–G3) before any real player is involved.
8. **A study with real players**, each playing both versions (a trial with 5 players, then about 24 players), to check G5–G7.

1.3 Hypotheses

Table 1: Hypotheses and how each is tested

ID	Hypothesis	Test
H1	The learner finds the best attack type for a situation when a best one exists.	With computer players, on a test problem with a known answer: it picks the best type at least 85% of the time within 300 games (G1).
H2	A reward with partial scores teaches the learner faster than a simple yes/no reward.	With computer players, 20 repeated runs: number of games needed to reach the G1 level (G3).
H3 (main)	Players change their base more with the adaptive director than with the normal waves.	Each player plays both versions; building spread per session; Wilcoxon signed-rank test, $p < .05$ and effect size $dz \geq 0.5$ (G5).
H4	Players do not find the director less fair.	The fairness rating is not worse by more than 0.5 on a 7-point scale (G6).
H5	Players notice that the enemies adapt.	At least 70% say, without being asked directly, that the enemies adapted (G7).

1.4 Scope

In scope

- A single-player maze tower defense for Windows PC, played with mouse and keyboard, built in Unity 6.
- A 48 × 48 tile map made automatically from a random seed, with rocks, ore deposits, and one to three places where enemies enter each wave.
- Walls and towers that enemies can dig through: a path to the core always exists, and enemies decide by cost whether to walk around or dig through.
- An ore economy: collectors on deposits that run out, a build menu with fixed prices, research that unlocks tower upgrades, and a power rule that links towers to the core, collectors and pylons.

- Several kinds of enemies that the attack types can use: walkers, sappers (dig cheaply), flyers (fly straight over everything), raiders (attack the ore collectors) and siege enemies (shoot towers from out of range).
- The director, the test bench, the data recording, the replay tools and the analysis scripts.

Out of scope

- Making the game harder or easier as a goal. The difficulty is kept the same on purpose; a change that makes the game harder without changing how the player builds counts as a step backwards.
- Deep reinforcement learning, neural networks and behaviour written by large language models (LLMs): about 25 decisions per session is far too little data to train them, and they would make the game impossible to replay exactly (Section 2.4).
- Multiplayer, mobile versions, in-game purchases and final art. Art, sound and the look of the screens are only made good enough for the study.

Limitations and requirements

- The study version runs on 64-bit Windows using Unity's Mono runtime, because that is where the repeatable game core was tested. A check in the build process refuses any other target.
- Minimum computer: one that runs Unity 6 at 1920 × 1080 and 60 frames per second (a four-core processor, 8 GB of memory, a DirectX 11 graphics card).
- The director decides once per wave, so it adapts from wave to wave, not second by second.
- The settings that sort situations into groups, and the definition of the main measurement, are fixed before the trial study. After the final build is fixed, only crash fixes are allowed.

1.5 How to Read This Document

The gameplay (screens, controls, towers, enemies and economy) is described in the separate Game Design Document. This proposal only covers it where the technical design needs it. Names from the program code are written in `this font`. The project's fixed rules are numbered I1–I12 (Table 6) and the checkpoints are numbered G1–G7 (Table 9). **Technical words are explained in the glossary at the end of this document (Appendix A).**

2. Literature Review

2.1 Adaptive Game AI

Dynamic difficulty adjustment. Hunicke (2005) argued that a game should adjust its difficulty to the player while it is being played, and should do it without the player noticing. Yannakakis and Togelius (2018) describe this work as part of *player modelling*: most systems guess the player's skill from results (health lost, time taken, wins) and then move a difficulty setting toward a target. Sutoyo et al. (2015) used this in a tower defence game, changing the difficulty based on the player's remaining lives, the enemies' health, and the skill points the player chose. This project also decides once per wave, like them, but it does not change the difficulty: it keeps the difficulty fixed and changes what the wave is made of.

AI directors. The director in *Left 4 Dead* estimates how stressed each player is, and then increases, holds, or reduces the number of enemies to create exciting ups and downs (Booth, 2009). It follows fixed rules and does not learn about the individual player. It separates "what to send" from "how the enemies behave", which is the same split this project uses (Section 3.4).

Dynamic Scripting. Spronck et al. (2006) change how computer opponents behave during play: each rule in a list has a weight, and after each fight the weights of the rules that were used go up or down depending on the result. This is the work most similar to this project. The differences are: this project uses probabilities that are updated from evidence instead of simple weights; it keeps separate results for each type of player base instead of one list for everyone; and its goal is to make players vary their bases, not to balance difficulty.

2.2 Related Works

Table 2: Comparison with related adaptive systems

System	What it changes	Learns about each player	Goal	Keeps difficulty the same
Hunicke (2005), DDA	Items and enemy strength	No (fixed rules)	Keep the player at a target difficulty	No: difficulty is what it changes
Left 4 Dead director (Booth, 2009)	Number of enemies and pacing	No (fixed rules)	Exciting pacing	No
Dynamic Scripting (Spronck et al., 2006)	Weights of opponent rules	Yes, one list for everyone	Win rate and balance	Partly
Sutoyo et al. (2015), DDA in tower defence	Lives, enemy health, skill points	No	Match difficulty to skill	No
This project	The *kind* of wave: enemy types, entry points, targets	Yes, a separate score per situation and attack type	Player changes the base	Yes: same enemy strength (I2)

Two more works shape how the project is tested rather than what is built. Al Enezi and Verbrugge (2023) let players face several different guard patrol behaviours in a stealth game and asked whether players could tell them apart in difficulty and enjoyment; mostly they could. This is the closest example for the study here, which also asks whether players notice the adaptation (G7)

and how they rate it (G6). To check whether the learner's probabilities are realistic, the project uses the Brier score (Brier, 1950) and the expected calibration error as defined by Guo et al. (2017).

2.3 Algorithms and Techniques

2.3.1 Finding paths with a flow field

Many tower defense games find a separate path for each enemy, often with the A-star method. When hundreds of enemies go to the same target, it is cheaper to calculate one *flow field* instead: starting from the target, every tile on the map stores how far it is from the target and which neighbouring tile to step to next (Emerson, 2019). Each enemy then simply follows the arrows, which takes almost no work per step.

This project builds the flow field by spreading outward from the core over the whole map, like a flood, always keeping the cheapest cost found for each tile. This is the queue-based version of the shortest-path method of Bellman (1958) and Moore (1959), often called SPFA. Moving to a neighbouring tile costs 10 (straight) or 14 (diagonal), multiplied by how expensive the tile is to cross. The unusual rule is that walls and towers **do not block** the path. They are expensive ground that can be destroyed: a wall costs 15, 30 or 45 depending on its strength, an ore collector 20 and a mountain 60, and towers stand on walls or rocks. An enemy standing on such a tile chews through it. Because every tile can be crossed at some cost, there is always a path to the core, and enemies choose between "walk around" and "dig through" by comparing costs, without any special code for that choice. One more rule stops enemies from slipping diagonally between two solid tiles.

$$cost(n) = \min \text{ over neighbours } m \text{ of } [\, cost(m) + step(n, m) \times terrain(n) \,], \quad step \in \{10, 14\}$$

The same method builds four flow fields: one for walkers; one for sappers, in which the player's walls cost only a fifth (mountains keep full price, so sappers do not dig through the same rock every time); one that leads raiders to the player's ore collectors; and one that pulls enemies toward towers with the "lure" ability. All four are rebuilt together after every change. Flyers do not use the fields and fly straight. Rebuilding the walker and sapper fields took about 0.3–0.6 milliseconds without Unity and 0.8–1.1 milliseconds inside the Unity editor (measured on a 38 × 38 map with up to a quarter of it built). This is so fast that more complicated versions (splitting the map into pieces, or using background threads) were removed. The four fields on today's 48 × 48 map will be measured again as part of checkpoint G2.

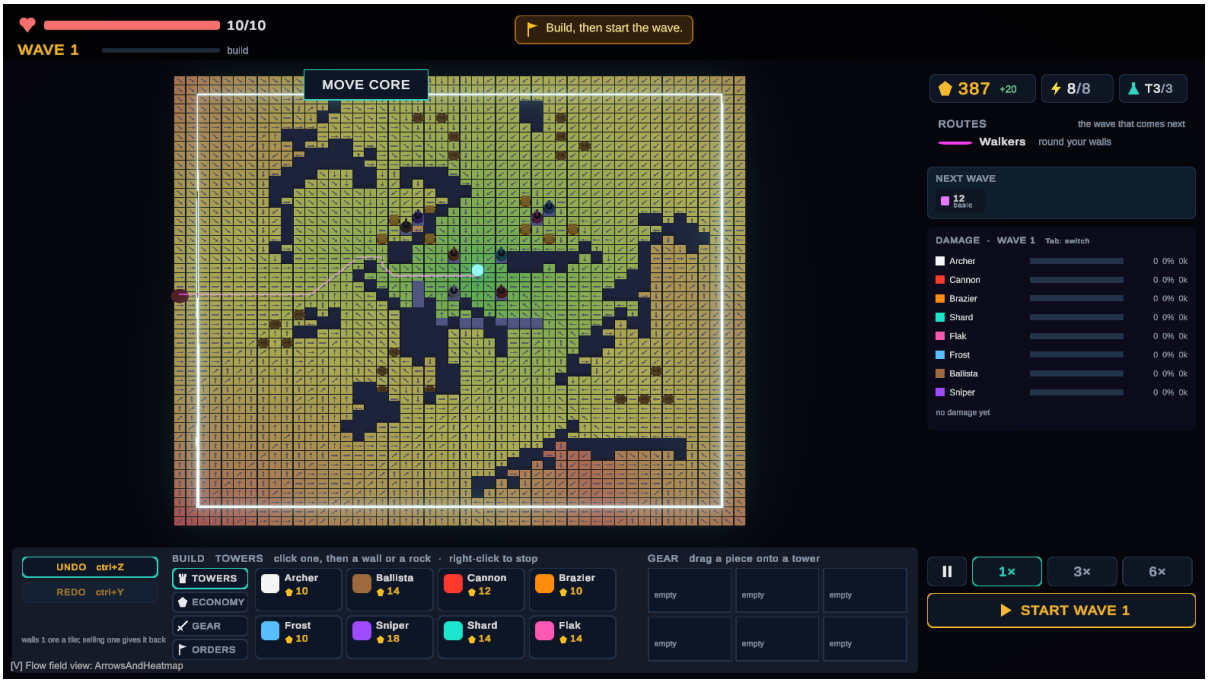


Figure 1: The flow-field view on a 48 × 48 map made from a seed: the colour shows the cost to reach the core (green is near, red is far); dark tiles are rocks; the pink line is the path walkers will take.

2.3.2 A repeatable game

A learning director has to be tested on replays and on about a million practice games, and the study has to show that both versions of the game sent comparable waves. Both need the same input to always give the same game. Unity's normal game loop does not do this: results change with the frame rate and the speed setting, the order in which objects update is not defined, and timers jump to the next frame. So the game rules were moved out of Unity's scripts into a separate C# program that moves forward in fixed steps (ticks) of 0.02 seconds. The methods used are standard, but all of them must work together:

- **Seeded random numbers.** The PCG32 random number generator (O'Neill, 2014), with a separate stream for each purpose (map, deposits, waves, director), so that adding a new use of randomness never changes the existing ones.
- **Maths that gives the same answer everywhere.** Some built-in maths functions (logarithm, exponent, power, sine and cosine) can give slightly different results on different computers, so the project uses its own versions. Long calculations are rounded at every step so the result is identical everywhere.
- **A fingerprint of the game state.** A 64-bit FNV-1a hash (Noll et al., 2026) of the whole game state after every tick. A replay stores the seed, every player command with its tick, and these fingerprints, so any difference is found immediately.
- **Hits decided in whole ticks.** A tower's hit is applied at once or after a delay fixed when it fires; the flying projectile is only drawn on screen (rule I12).

2.3.3 Learning which attack works: contextual bandits and Thompson sampling

Choosing an attack type each wave, seeing the result, and choosing again is called a *contextual bandit* problem. The name comes from slot machines ("one-armed bandits"): there are several options to choose from (the attack types), a situation to look at first (the player's base), and a reward that depends on both. *Thompson sampling*, first proposed by Thompson (1933), picks an option by drawing a random "guess" of how good each option is from what has been learned so

far, and taking the best guess. Options it is unsure about therefore get tried sometimes, and options that are clearly good are picked most of the time, without any extra setting to tune (Russo et al., 2018). Chapelle and Li (2011) showed with simulated and real data that it works very well compared with other methods, and Agrawal and Goyal (2013) proved how well it performs when the reward follows a straight-line formula. A well-known straight-line method is LinUCB (Li et al., 2010), which assumes that the reward of each option rises or falls evenly with the situation. This project does not use that assumption, because here the reward can jump suddenly at a *threshold*: a maze is either long enough to make sappers worth sending, or it is not. A straight line cannot show such a jump, and an open-source bandit program that was examined failed in exactly this way (katopz, n.d.).

The learner therefore sorts situations into **groups (bins)**: one measurement of the base is divided into 5 ranges, and for every range and attack type the learner keeps its own score table, a Beta distribution with two counts, α (roughly the successes) and β (roughly the failures). That makes 5 × 4 = 20 tables. Random draws from these tables are made with the method of Marsaglia and Tsang (2000). When a table has very little data, the learner borrows from similar situations seen recently, using a Nadaraya–Watson weighted average (Nadaraya, 1964; Watson, 1964) over the last 128 results, where more similar situations count more:

$$\hat{Q}(x, a) = \Sigma_i K(x, x_i)\, r_i \,/\, \Sigma_i K(x, x_i), \quad \alpha_0 = 1 + k\cdot\hat{Q}, \quad \beta_0 = 1 + k\cdot(1 - \hat{Q})$$

The player learns while the director learns, so old results become less useful over time. The learner therefore slowly forgets: before each update, the counts shrink back toward the starting point Beta(1, 1), similar to the forgetting used in bandits for changing situations (Garivier & Moulines, 2011). A reward between 0 and 1 is turned into a success or failure by a weighted coin flip, which keeps the counting exact:

$$\tilde{r} \sim Bernoulli(r), \quad \alpha \leftarrow 1 + G(\alpha - 1) + \tilde{r}, \quad \beta \leftarrow 1 + G(\beta - 1) + (1 - \tilde{r}), \quad G = 0.95$$

2.3.4 Measuring how much players rebuild

The main measurement is the Shannon entropy (Shannon, 1948) of where the player builds, which measures how spread out the building is. The map is divided into a 6 × 6 grid of regions. Every tile the player builds after wave 1 (walls and towers, counted even if they are later sold or destroyed) is counted in its region:

$$H = -\Sigma_r\, p_r \ln p_r \,/\, \ln K$$

Here p_r is the share of built tiles in region r, and K is the number of regions where building is possible. H = 0 means everything was built in one region (repairing the same spot); H = 1 means building was spread evenly. It is calculated afterwards from the saved data, so its definition can still be changed for free until it is fixed before the trial study. Its known weakness is that a player who slowly grows one maze across the whole map also gets a high score; this is checked with two other numbers, the change in the base after each wave and the number of tiles and towers sold.

2.4 Approaches Considered and Rejected

Table 3: Rejected approaches and the reason

Approach	Why it was rejected
Straight-line contextual bandit (LinUCB; Li et al., 2010)	Cannot show the sudden jumps that this game is full of; the failure was seen in an examined program (katopz, n.d.).

Approach	Why it was rejected
Deep reinforcement learning / neural networks (Sutton & Barto, 2018)	About 25 decisions per session is far too little data to train one.
ε-greedy exploration (Sutton & Barto, 2018)	Thompson sampling tries uncertain options in proportion to how unsure it is, and has no extra setting (ε) to tune (Russo et al., 2018).
Behaviour written by LLMs or online decision services	Cannot be replayed exactly (no control over randomness, can change remotely) and do not learn about the individual player.
Walls that fully block the path	Need extra code to check and undo placements; walls that can be dug through always leave a path and removed a whole group of bugs.
Splitting the flow field into pieces or background threads	A full rebuild already takes about a millisecond; the extra complexity gave nothing measurable.

2.5 Tools for Development

Table 4: Development tools

Tool	Use
Unity 6 (6000.4.10f1), URP (Unity Technologies, 2026)	Shows the game, reads the mouse and keyboard, draws the screens, and makes the study build.
C# / .NET 9 (Microsoft, n.d.)	The game rules, the director and the test bench, which run both inside Unity and on their own.
NUnit (NUnit Project, n.d.) and the Unity Test Framework	One set of automatic tests (661 at the time of writing), run both in Unity and with `dotnet test`.
Thesis command-line tool (made in this project)	Runs games without Unity: `run`, `replay`, `pin`, `telemetry`, `entropy`, `balance` and `bench`.
Git (Git, n.d.) and GitHub	Keeps every version of the code; every result file records the exact version and command that produced it.
Python (Python Software Foundation, n.d.)	Analysing the game data and the study results.
Graphviz (Ellson et al., 2002)	Drawing the diagrams in this document.

3. Deliverable Details

3.1 Project Overview

The game is a single-player maze tower defense on a 48 × 48 map made from a random seed. The camera looks down at the board at a 52° angle. The player places the core, builds ore collectors on deposits to earn ore (each deposit holds 30 ore, then runs out), draws walls of three strengths to make the enemies' path longer, and places towers on walls or rocks to shoot the enemies. Everything is bought from a build menu at fixed prices. Research unlocks tower upgrades, and a tower only shoots while it gets power from the core, a collector or a pylon. Enemies enter from one to three edges each wave and follow the flow field to the core; the game is lost when enough of them reach it. The full design is in the GDD. Figures 2–5 are pictures of the current game.

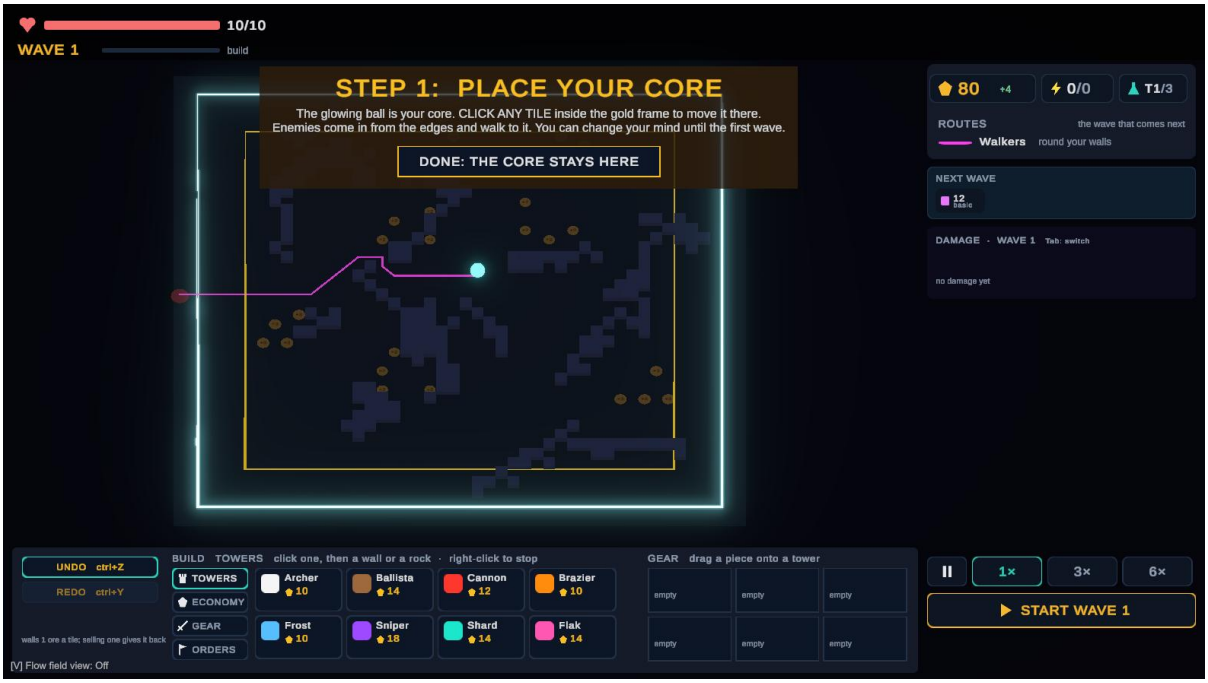


Figure 2: Start of a session: the player chooses where the core goes, inside the gold frame. Brown discs are ore deposits, dark tiles are rocks; the pink line is the path of the first wave.

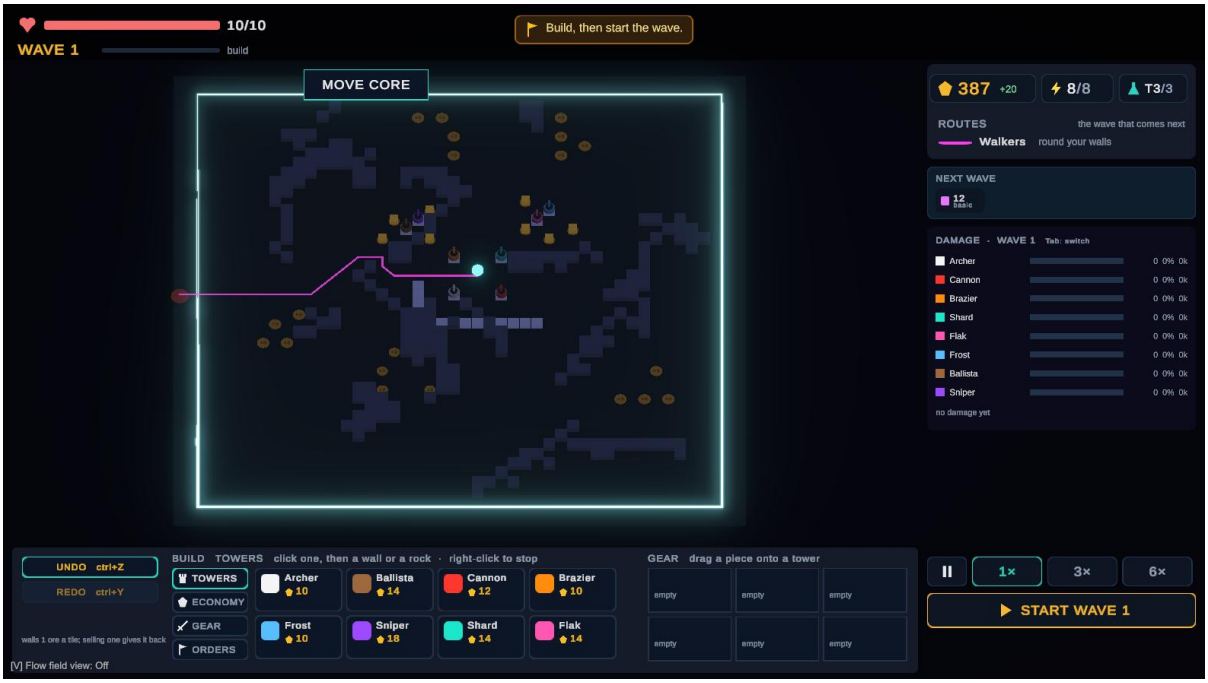


Figure 3: Building between waves. Towers stand on walls around the core and the collectors. The right panel shows ore, powered towers, research level, the next wave, and how much damage each tower did last wave.

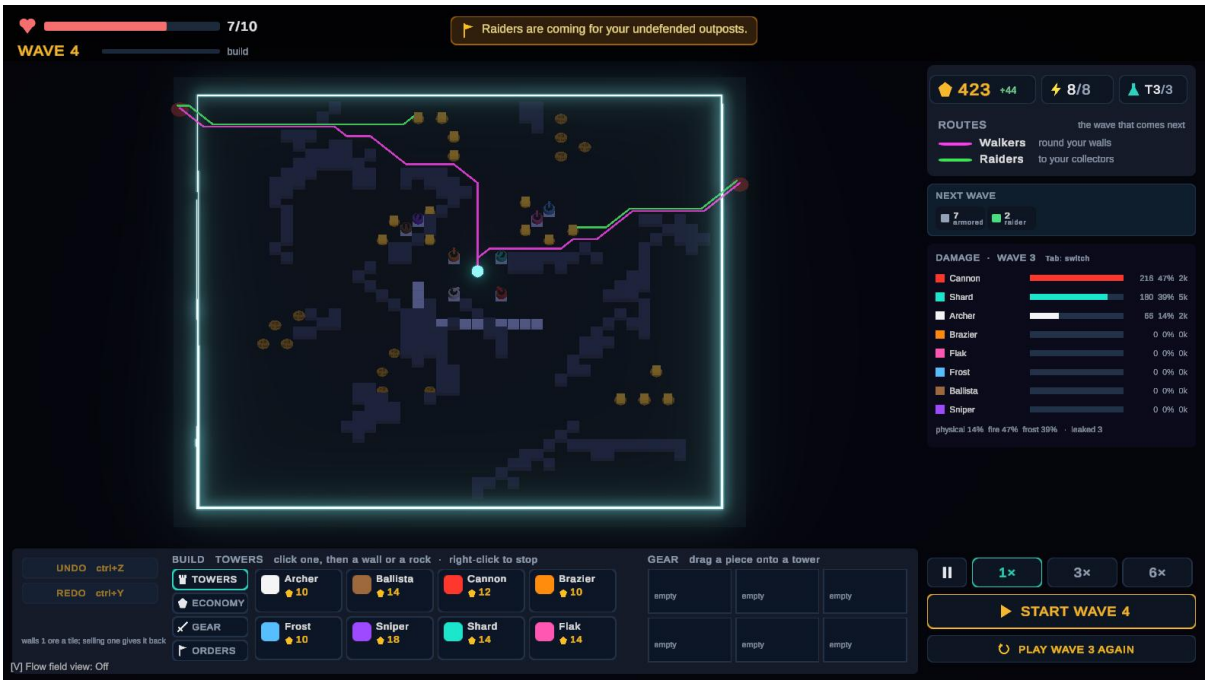


Figure 4: The director at work (development version). The player left collectors unprotected, so the next wave is planned as a raid: the banner warns about it, the icons show 7 armoured enemies and 2 raiders, and a second path line (green) shows where the raiders will go.

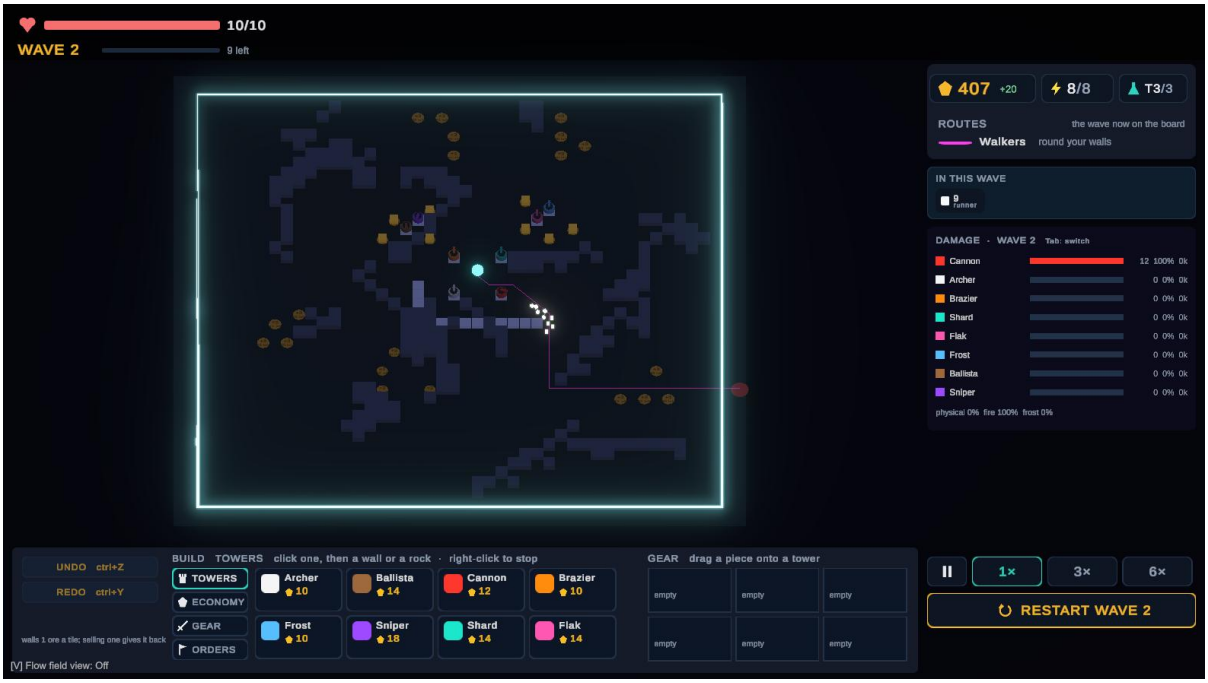


Figure 5: A wave in progress: fast enemies follow the path past the wall line while the cannon shoots at them; the damage panel fills up during the wave.

3.2 System Architecture

The code is split into two parts: a pure C# core that holds all the game rules, and a thin Unity layer around it (Figure 6). Every game rule lives in `Assets/Thesis`, which is not allowed to use Unity at all (rule I3, checked automatically). The Unity scripts only turn clicks into commands, run the core in fixed steps, and draw what the core reports. Because the core does not need Unity, the same code also runs on its own, which is how the tests, the replay tools and the test bench work.

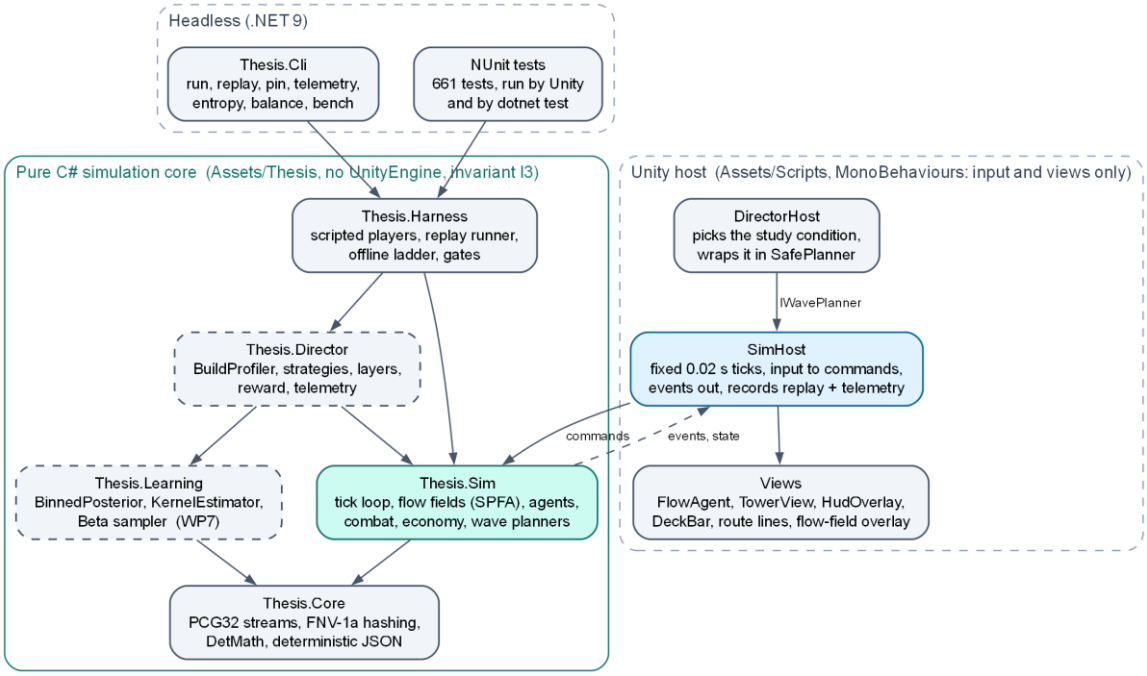


Figure 6: The parts of the program and how they depend on each other. Dashed boxes are designed but not built yet (Thesis.Learning, Thesis.Director).

Table 5: Size of the code at the time of writing

Part	Lines of C#	Contents
Game core (`Assets/Thesis`)	≈ 13,700	Core, rules, data recording, test bench
Unity layer (`Assets/Scripts`)	≈ 10,000	Game host, drawing, screens, editor tools
Tests (`Assets/Tests`)	≈ 15,100	661 tests, run in Unity and on their own

3.3 Game Core

`Simulation.Tick()` moves the whole game forward by one fixed step, always doing things in the same order, and the tests check that order (Figure 7). Player input never changes the game directly: a click becomes a command marked with the tick at which it happens. Whether something may be built is decided by one function, `Placement.CanPlace`, which the building preview also uses, so the preview can never disagree with a real click (rule I9). The speed buttons only change how many ticks run per screen frame (at most 40), never what a tick does.

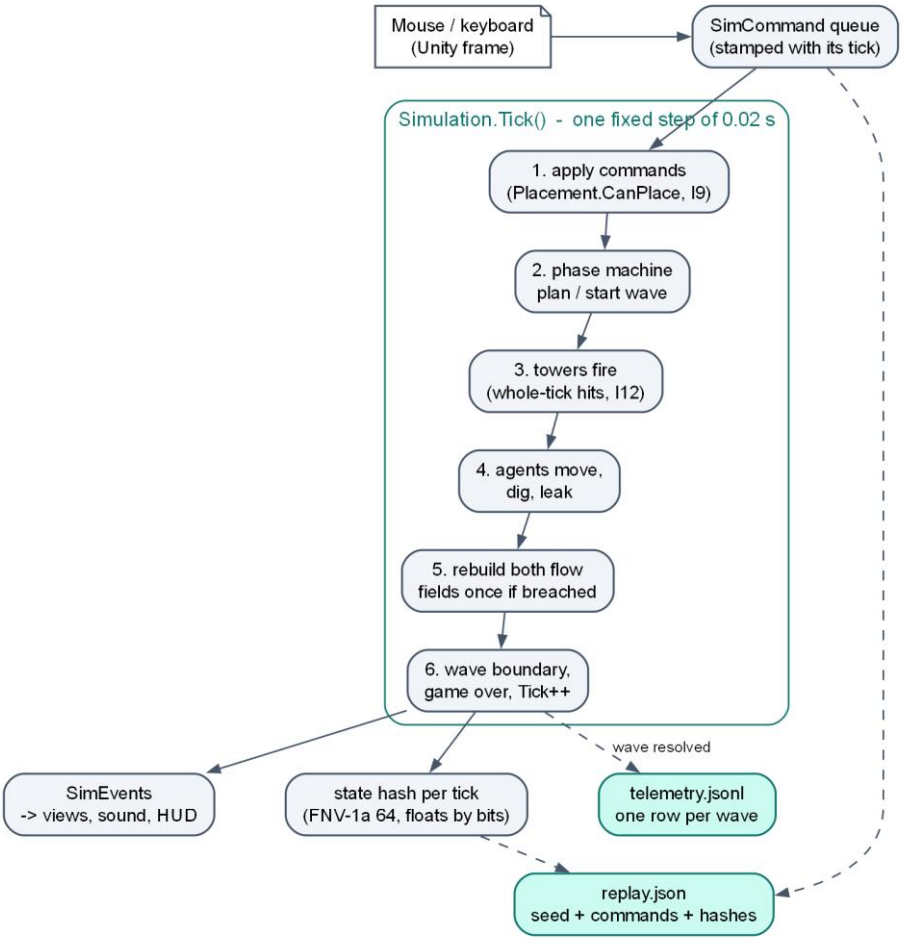


Figure 7: What happens in one game step, and the two files a session leaves behind.

The rules that the whole design depends on are written down and checked by tests (Table 6).

Table 6: Fixed rules of the project

#	Rule	Why
I1	The same map, seed and input always give the same game; fixed time steps; only the project's own random number generator.	Replays, automatic tests, and a fair comparison of the two versions.
I2	Every attack type costs the same enemy strength at the same wave number.	Otherwise "the adaptive version was harder" would explain the result.
I3	The learner and the game rules do not use Unity.	So they can run a million practice games and be tested on their own.
I4	The learned part only adds a correction of at most ±1 to a hand-written score.	Safe when the learner knows nothing yet; and the version without learning comes for free.
I5	Fixed rules remove a choice (score −∞) before the learner is asked.	What is allowed and fair is decided above learning.
I6, I7	Settings are fixed before the trial study; no code changes after the final build, except crash fixes.	Every player plays the same game.
I8	The director never runs every frame; at most 16 ms per wave.	The player never sees it slow the game (G2).
I9	`Placement.CanPlace` alone decides what may be built.	Preview, click and test bench always agree.
I10	Anything random offered to the player depends only on (seed, wave, number of rerolls). Not in use: the current game has no random shop.	Both versions would see the same offers.
I11	The director only shapes waves; never prices, tower stats or the player's ore. Open problem: raiders and siege enemies affect income and towers (Section 4.4).	The player's power is the same in both versions.
I12	Tower hits are decided in whole ticks; projectiles are only drawn.	The frame rate never affects the rules.

The repeatable core was tested on 2 October 2026 by recording games inside Unity and without it, and replaying each one on the other: a 25-wave game of 111,861 ticks with 466 player commands matched at every single tick in both directions. Games recorded through Unity's real game loop also matched, at speeds from x1 up to the 20-ticks-per-frame limit, with pauses, and when quit in the middle of a wave. Ten saved replays are played again by every test run, so any change that alters the rules is noticed immediately.

3.4 How a Wave is Planned

Everything that decides a wave sits behind one interface, so the normal waves, the development director and the final director can be swapped for one another as the study versions:

```
public interface IWavePlanner {
    string Name { get; }                          // saved in the data as the version
    WavePlan PlanWave(WaveContext context);       // may only read; the game state is checked
afterwards
    void OnWaveResolved(WaveOutcome outcome);     // receives a copy of the result
}
```

The next wave is planned when the building time starts, so the player can see it (the enemy icons and the warning) and respond before pressing Start. The game takes a fingerprint of its whole

state before and after `PlanWave` and stops with an error if the planner changed anything. `ValidatePlan` checks that the plan is sensible. Every planner also runs inside a `SafePlanner`, which falls back to the normal waves and logs an error if the planner fails, so a planner bug can never break a session. The normal waves (`EscalationPlanner`) get more enemies, more speed and more health every wave; this is the comparison version.

A first, rough version of the hand-written step (`RoughDirector`) is already in the game (Figure 4). It looks at the base (groups of towers, unprotected collectors, the main damage type, how much damage can hit flyers, how long the maze is) and gives scores to raids, siege enemies, resistant enemies, flyers and sappers, with a small penalty for repeating the last choice. It is used for play-testing and is what the current game runs. It does not learn and has no shared enemy price list yet, so it does not meet rule I2, and it is not one of the study versions.

3.5 Adaptive Director

The director runs once per wave (Figure 8). It saves every value it used into that wave's line of data, so the question "why did it choose this at wave 7?" can be answered by reading one line.

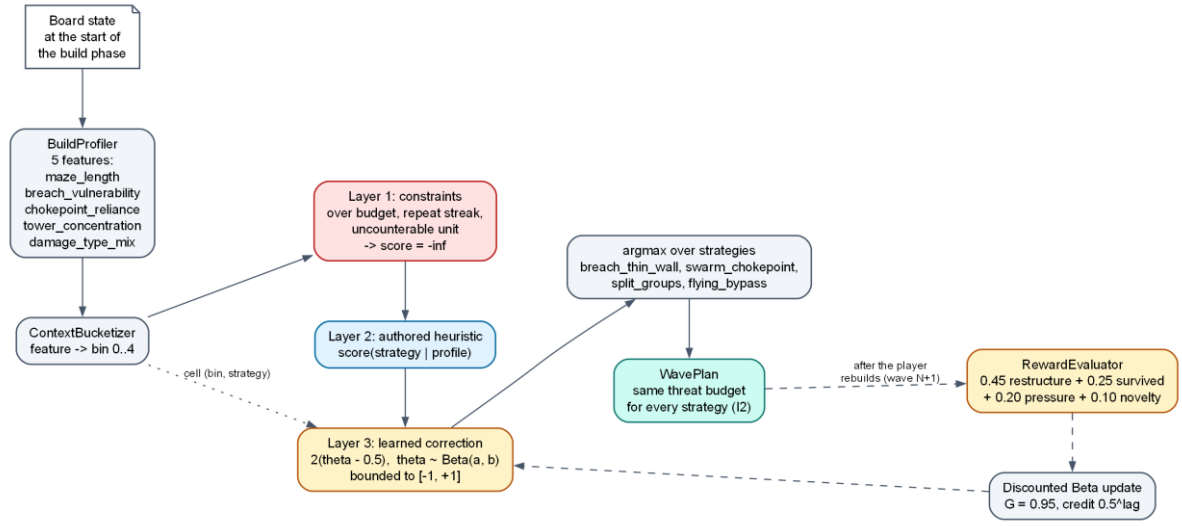


Figure 8: The steps of the director. The reward for wave N is only known when wave N+1 is planned, after the player has had time to rebuild.

3.5.1 Base analysis

Table 7: The five measurements of the player's base (first definitions; fixed before the trial study)

Measurement	What it means
`maze_length`	How long the walkers' path is, compared with the same path on an empty map.
`breach_vulnerability`	How much of the path's cost comes from tiles that can be dug through: how much the path depends on walls holding.
`chokepoint_reliance`	Whether the towers' damage is done on a few tiles of the path or spread along it.
`tower_concentration`	Whether the towers stand close together or spread out (average distance from their centre, compared with the map size).
`damage_type_mix`	Whether last wave's damage was mostly one type (physical, fire or frost) or a mix.

The measurements describe what the player *chose* to build. An earlier version of the game sold towers from a random shop, which could push players to rebuild in both versions; rule I10 made

those offers depend only on the seed. The current game uses a fixed build menu instead, so the only luck left is the map, its deposits and the entry points. All of them come from the seed, and both versions use the same sets of seeds (Section 4.3).

3.5.2 Attack types and the shared enemy price list

Each attack type turns an amount of enemy strength and the current situation into a wave. The planned types are `breach_thin_wall` (sappers at the thinnest wall), `swarm_chokepoint` (many weak enemies through the gap the player relies on most), `split_groups` (enemies from several sides at once) and `flying_bypass` (flyers over the maze). The strength allowed for wave w is the price of the normal wave w on one shared price list, and any plan that misses it is refused:

$$Budget(w) = Price(EscalationPlanner.Plan(w)), \quad |ThreatSpent(plan) - Budget(w)| \le \varepsilon$$

Only the shared price list is ever tuned, never a single attack type, so rule I2 cannot slowly break. The final set of attack types is chosen in work package WP9; the development director's types (raid, siege, resistant enemies, air raid, maze breach) are candidates together with the four above.

3.5.3 The three steps of a decision

- **Step 1, fixed rules:** an attack type is removed if it costs too much, has been used too often in a row, sends an enemy the player cannot fight (flyers when no tower can hit them), or is too similar to the last wave. A removed type gets the score −∞ and the learner is not asked about it (rule I5).
- **Step 2, hand-written score:** each remaining type gets a score from the five measurements. This step works as a complete director on its own, and is used as the version without learning.
- **Step 3, learned correction:** for each remaining type, a random guess θ is drawn from its score table and turned into a correction 2(θ − 0.5), which lies between −1 and +1. While a table has no data the correction is 0. The correction can change the order of types that scored about the same, but it can never choose on its own (rule I4).

```
foreach (var a in survivingStrategies) {
    float theta = SampleBeta(alpha[bin, a], beta[bin, a]);
    float corr  = IsCold(a) ? 0f : 2f * (theta - 0.5f);   // between -1 and +1
    score[a]    = Heuristic(a, profile) + corr;
}
Play(ArgMax(score));                                      // ties: the lowest number wins
```

Every chosen attack is announced before the wave with one hand-written line, chosen by situation group and attack type. The lines are written in advance, never generated. They make the game feel responsive, and they are also how the study checks that players noticed the adaptation (G7).

3.5.4 Reward

$$r = 0.45 \cdot restructure + 0.25 \cdot survived + 0.20 \cdot pressure + 0.10 \cdot novelty$$

`restructure` is how much the player's base changed after the wave (measured on the five numbers, with a maximum). `survived` is whether the core is still standing; this stops an attack type from scoring well by ending the game, because a player who has lost builds nothing more. `pressure` is the share of the wave that got close to the core, and `novelty` is how different the wave was from the one before. A player may be reacting to the last two or three waves, so credit for a change is shared backwards, halving for each wave further back. A simple yes/no reward (the base changed enough, and the core survived) is also built, to compare against at checkpoint G3. The learner's tables can be saved and loaded, so a director trained on one player can later be tested on another (the transfer test).

3.6 Saved Data and Replays

Every session writes two files. `replay.json` holds the seed, the settings, every player command with its tick, and the state fingerprints; it is the exact record of the game. `telemetry.jsonl` holds one line per wave: what was sent, what happened (enemies spawned, killed and leaked, damage by type, core health), what the player built, bought and sold around it, and, once the director exists, its measurements, scores, removed choices, reward and time taken. A line contains nothing that the game rules do not decide (for example, no clock time), and numbers are written the same way on every computer, so the data file can be made again from the replay, byte for byte. The file format has a version number, and lines are only ever added, never changed.

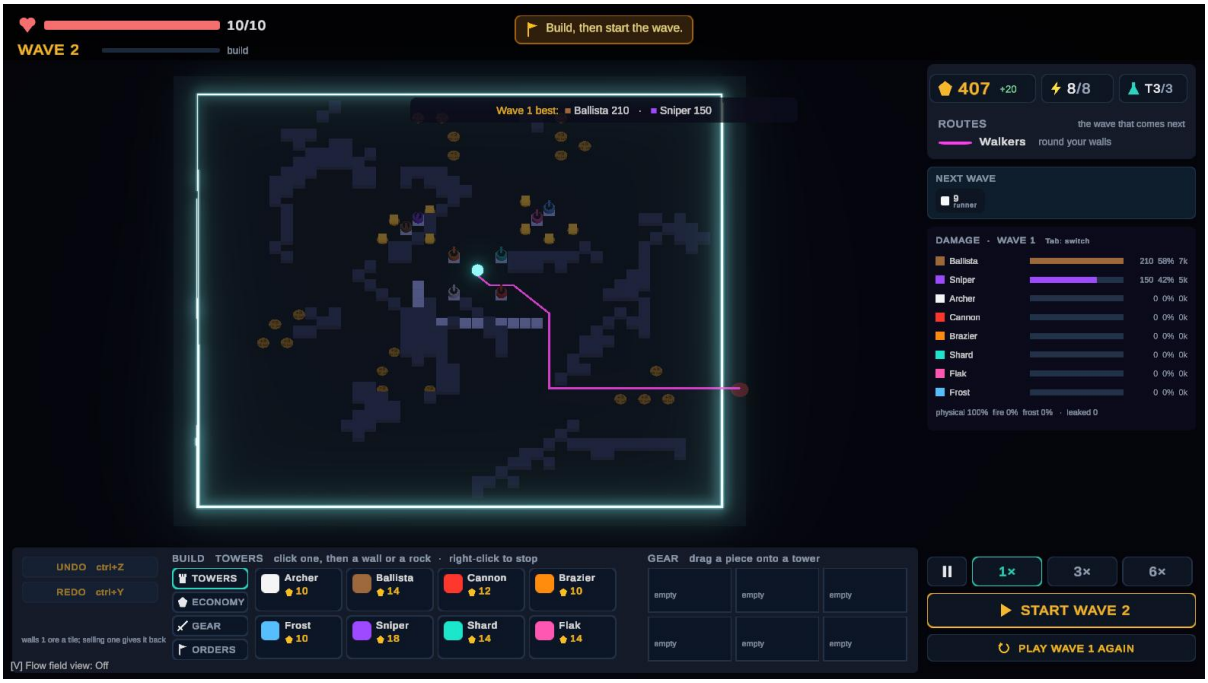


Figure 9: After a wave: the damage panel shows each tower's share of the damage, and the next wave is already planned from a different side. The same numbers are saved in the data file.

The saved data is already used for balancing the game. In a 77-wave session by a tester on 28 October 2026 it showed that the core was not under any pressure for 66 waves, that the economy grew out of control (28 collectors, up to 877 ore saved), that enemy health growing by 8% a wave made the game suddenly impossible at the end, and that the development director stopped reacting after wave 11. Each finding led to a change the same day (deposits that run out, a steadier health curve, and a director that checks every weak point), and the next test session will check them.

3.7 Current Status

Table 8: What is built and what is planned

Part	Status	Evidence
Repeatable game core, replays, command-line tool	Done	Test result of 2 October 2026; saved replays checked in Unity and on their own
Combat, enemy movement types, economy	Done	8 towers and a pylon, 12 enemy types, ore deposits, research
Saved data and the main measurement	Done	`thesis entropy`; versioned file format

Part	Status	Evidence
Unity screens and path lines	Done, still improving	Test sessions on 12, 14, 20 and 28 October 2026
Development director (rough version of step 2)	Done	`RoughDirector`, Figure 4
Learning part (WP7)	Planned	Test problem with a known answer
Base analysis, attack types, price list (WP8–WP9)	Planned	Needs the final definitions of the five measurements
Decision steps, full director, test bench, checkpoints (WP10–WP13)	Planned	

4. Evaluation Methodology

4.1 Checkpoints

Each checkpoint has a pass number and a fail number that are fixed before the work is done, and the fail side says what will be done next. This way a failure leads to a clear decision instead of a changed target.

Table 9: Checkpoints

Checkpoint	Pass	Fail → action
G1 the learner works	Picks the best attack type at least 85% of the time within 300 games, and does better than random choice and taking turns	Below 70%, or no better than those → a bug, or the measurements carry no useful information: revisit the measurements
G2 no visible slowdown	The director takes at most 16 ms per wave	A frame takes more than 33 ms → split the work over several frames
G3 partial reward is better than yes/no	Reaches the G1 level in fewer games than the yes/no reward, over 20 repeated runs	No difference → use yes/no and report it
G4 enough data per group	Trial with 5 players: in a typical session, at least 8 results per group that was used	Fewer than 4 → use 3 groups instead of 5, and report it
G5 players rebuild more (main)	About 24 players, each playing both versions; Wilcoxon test $p < .05$ and effect size $dz \geq 0.5$	$p > .05$ or $dz < 0.2$ → report that it did not work
G6 not unfair	The fairness rating is not worse by more than 0.5 on a 7-point scale	Worse → the reward or the warning text is at fault
G7 players notice	At least 70% say, without being asked directly, that the enemies adapted	Below 50% → G5 cannot be interpreted until this is fixed

Passing G1 does not mean G5 will pass: a learner can work perfectly with computer players and still change nothing about how a real person plays. If that happens, that is itself the finding.

4.2 Testing With Computer Players

Before any real player takes part, the test bench lets the director play against computer-controlled players with fixed habits (one that does nothing, one that builds long detours, one that guards, one that sells, and mixes of these), without Unity. The expected order of results is: random choice < normal waves < hand-written director < full director, measured by the reward and by how spread out the computer players' building becomes. If the full director does not beat the hand-written one, this must be found in semester 1, not during the study. G1 and G3 use a test problem in which the best attack type switches at a known threshold, so the right answer is known; a million games of it run in seconds. A calibration report (Brier score and expected calibration error, which check whether the learner's probabilities match what really happened) is published together with G1.

4.3 Study With Real Players

Each player plays two sessions: one with the normal waves and one with the adaptive director. The order of the versions and the map seed are balanced using two sets of seeds, so no player sees the same map twice, and each version is played on each seed and in each position equally often. (The development build picks a new random seed every session; the study build will use the fixed sets.) Both sessions of a player are cut to the same number of completed waves before

comparing them. Measurements: building spread (the main one), the change in the base after each wave, tiles and towers sold per wave, and the same spread on other grids (4 × 4, 8 × 8, walls only, towers only), all fixed before the study. There is also a questionnaire about fairness (G6), tested with a non-inferiority test (Walker & Nowacki, 2011), and an open question about whether the enemies adapted (G7). The results are compared with the Wilcoxon signed-rank test on each player's two sessions (Wilcoxon, 1945), and the size of the difference is reported as dz, the effect size for paired data (Lakens, 2013).

4.4 What Could Make the Results Wrong

- **Difficulty:** controlled by the shared enemy price list (rule I2) and by the director never changing the player's power (rule I11).
- **Map luck:** the map, its ore deposits and the entry points come from the seed; both versions use the same seed sets in a balanced order, so neither version gets easier maps.
- **Income:** raiders destroy collectors and siege enemies switch towers off, so an attack type that sends them changes the player's income and fire power, which the normal waves do not (rule I11, open problem S14). Before the trial study they must either be counted in the price list or be sent in the same way in both versions.
- **Learning and order:** the order of the versions is balanced, and because every player plays both, each player's own style does not affect the comparison.
- **What the measurement really measures:** building spread shows *where* building happens, not directly how much the base changes; it is checked with the change in the base after each wave and with the number of sales.
- **Tuning after seeing the results:** the group boundaries and the definition of the measurement are fixed before the trial study (rule I6), and the game is fixed before the main study (rule I7).

5. Plan of Action

5.1 Tasks

Table 10: Work packages

Package	Content	Status
WP0–WP5	Move the game rules into a repeatable C# core; connect Unity to it; replays and the command-line tool; prove the game is repeatable	Done (2 Oct 2026)
WP-H	Fixes after the first review: safe planners, repeatable maths, saved replays in the tests	Done (2 Oct 2026)
WP-C1–C5	Combat, enemy movement types, shop (later replaced by a build menu) and saved data, tower screens, first set of towers and enemies with a balance check	Done (12 Oct 2026)
Gameplay rounds	Changes after play tests: generated map, ore economy, build menu, research, power, wall strengths, breakable mountains, upgrade paths, development director	Ongoing (to 31 Oct 2026)
WP7	Learning part: random draws, score tables, borrowing from similar situations, test problem, calibration check	Planned
WP8	Base analysis (the five measurements)	Planned
WP9	Attack types, shared enemy price list, several entry points	Planned
WP10	Fixed rules and hand-written scores; a director without learning	Planned
WP11	Reward, shared credit, the full director	Planned
WP12	Test bench: computer players and the expected order of results	Planned
WP13	Checkpoints G1–G3	Planned
WP14	The director on screen (warnings) and the study start screen	Planned

5.2 Schedule

Semester 1 has twelve weeks (W5–W16); week 5 began on 14 September 2026. Filled squares are done, empty squares are planned, and diamonds are checkpoints.

Table 11: Semester 1 schedule

Task	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15	W16
Repeatable game core, replays (WP0–WP5, WP-H)	■	■	■									
Combat, enemy types, shop, saved data (WP-C1–C3)			■	■								
Tower screens, first towers and enemies (WP-C4–C5)					■							
Gameplay rounds with testers					■	■	■					
Learning part (WP7)							□	□				
Base analysis, attack types (WP8–WP9)								□	□			
Decision steps, director on screen (WP10, WP14)									□			
Full director (WP11)										□		
Test bench (WP12)										□	□	
Checkpoints G1–G3 (WP13)											◆	
Report and study preparation												□

Table 12: Break and semester 2 schedule

Task	B1	B2	B3–4	W1–3	W4	W5	W6	W7–11	W12–13	W14–15
Trial study with 5 players, checkpoint G4	◆									
Fix the settings (cannot be undone)		◆								
Spare time			□							
Screen improvements (path preview, warnings)				□						
Balancing					□					
Second trial, questionnaire check						□				
Fix the final build (cannot be undone)							◆			
Main study, about 24 players (G5–G7)								◆		
Analysis									□	
Writing and releasing the work										□

If time runs short, work is cut in this fixed order: (1) automatically generated attack types (an extra goal), (2) the transfer test, (3) the second trial, (4) the number of players, from 24 to 18. The repeatable game core, the balancing and the final build are never cut.

5.3 Monitoring and Evaluation

Table 13: Input, process, output and benefit

Input	Process	Output	Benefit
Player commands (build, sell, upgrade, start) from mouse and keyboard; the map seed; the game version.	The game core plays each wave; between waves the director measures the base, removes unfair choices, scores and corrects, plans the next wave, and learns from the result.	A wave aimed at the current base and announced before it starts; a replay and one line of data per wave; the session's building spread.	Players keep making real decisions instead of repeating one layout; the thesis gets a repeatable and fair comparison of adaptive and normal waves.

Progress is followed per work package: each package has written goals, is checked against them, and ends with an entry in the development log. Every experiment is written up as a result file that states the expected outcome before the results, the date, the code version and the exact command to repeat it, including experiments that failed. Changes to the plan are written down as they happen; they will become the limitations chapter of the thesis.

5.4 Risks

Table 14: Main risks and how to handle them

Risk	Likelihood	How to handle it
Gameplay changes delay the director work	High	A fixed checkpoint; the learning part only needs the core and can be built at the same time; the cut order in 5.2
The learner is no better than the hand-written director (computer tests)	Medium	Found in semester 1; the hand-written director is a complete version on its own; report the result
Too little data per group (G4)	Medium	Borrow from similar situations; fall back to 3 groups
Waves of equal strength still feel unequal to players	Medium	Fairness checkpoint G6; only tune the shared price list
Raids and sieges change the player's income (rule I11)	High	Count them in the price list or send them the same way in both versions; decide with the supervisor before the trial study
A rule change breaks the repeatable core	Low	Saved replays are checked by every test run, in Unity and on their own; a fingerprint at every tick

6. References

Agrawal, S., & Goyal, N. (2013). Thompson sampling for contextual bandits with linear payoffs. In *Proceedings of the 30th International Conference on Machine Learning* (pp. 127–135). PMLR.

Al Enezi, W., & Verbrugge, C. (2023). Evaluating player experience in stealth games: Dynamic guard patrol behavior study. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 19*(1), 175–184. https://doi.org/10.1609/aiide.v19i1.27513

Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics, 16*(1), 87–90.

Booth, M. (2009, October 14–16). *The AI systems of Left 4 Dead* [Conference presentation]. Fifth Artificial Intelligence and Interactive Digital Entertainment Conference (AIIDE-09), Stanford, CA, United States. https://cdn.fastly.steamstatic.com/apps/valve/2009/ai_systems_of_l4d_mike_booth.pdf

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3.

Chapelle, O., & Li, L. (2011). An empirical evaluation of Thompson sampling. In *Advances in Neural Information Processing Systems 24* (pp. 2249–2257). Curran Associates.

Ellson, J., Gansner, E., Koutsofios, L., North, S. C., & Woodhull, G. (2002). Graphviz—Open source graph drawing tools. In P. Mutzel, M. Jünger, & S. Leipert (Eds.), *Graph drawing: GD 2001* (pp. 483–484). Springer. https://doi.org/10.1007/3-540-45848-4_57

Emerson, E. (2019). Crowd pathfinding and steering using flow field tiles. In S. Rabin (Ed.), *Game AI Pro 360: Guide to movement and pathfinding* (pp. 67–76). CRC Press.

Garivier, A., & Moulines, E. (2011). On upper-confidence bound policies for switching bandit problems. In J. Kivinen, C. Szepesvári, E. Ukkonen, & T. Zeugmann (Eds.), *Algorithmic learning theory* (pp. 174–188). Springer. https://doi.org/10.1007/978-3-642-24412-4_16

Git. (n.d.). *Git* [Computer software]. Retrieved October 28, 2026, from https://git-scm.com/

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 1321–1330). PMLR.

Hunicke, R. (2005). The case for dynamic difficulty adjustment in games. In *Proceedings of the 2005 ACM SIGCHI International Conference on Advances in Computer Entertainment Technology* (pp. 429–433). https://doi.org/10.1145/1178477.1178573

katopz. (n.d.). *katgpt-rs* [Computer software]. GitHub. Retrieved October 28, 2026, from https://github.com/katopz/katgpt-rs

Lakens, D. (2013). Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and ANOVAs. *Frontiers in Psychology, 4*, Article 863. https://doi.org/10.3389/fpsyg.2013.00863

Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010, April). A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web* (pp. 661–670).

Marsaglia, G., & Tsang, W. W. (2000). A simple method for generating gamma variables. *ACM Transactions on Mathematical Software, 26*(3), 363–372. https://doi.org/10.1145/358407.358414

Microsoft. (n.d.). *.NET documentation*. Retrieved October 28, 2026, from https://learn.microsoft.com/en-us/dotnet/

Moore, E. F. (1959). The shortest path through a maze. In *Proceedings of the International Symposium on the Theory of Switching, Part II* (pp. 285–292). Harvard University Press.

Nadaraya, E. A. (1964). On estimating regression. *Theory of Probability & Its Applications, 9*(1), 141–142. https://doi.org/10.1137/1109020

Noll, L., Vo, K., Eastlake, D., 3rd, & Hansen, T. (2026). *The FNV non-cryptographic hash algorithm* (RFC 9923). RFC Editor. https://doi.org/10.17487/RFC9923

NUnit Project. (n.d.). *NUnit* [Computer software]. Retrieved October 28, 2026, from https://nunit.org/

O'Neill, M. E. (2014). *PCG: A family of simple fast space-efficient statistically good algorithms for random number generation* (Technical Report HMC-CS-2014-0905). Harvey Mudd College.

Python Software Foundation. (n.d.). *Python* [Computer software]. Retrieved October 28, 2026, from https://www.python.org/

Russo, D. J., Van Roy, B., Kazerouni, A., Osband, I., & Wen, Z. (2018). A tutorial on Thompson sampling. *Foundations and Trends in Machine Learning, 11*(1), 1–96. https://doi.org/10.1561/2200000070

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal, 27*(3), 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

Spronck, P., Ponsen, M., Sprinkhuizen-Kuyper, I., & Postma, E. (2006). Adaptive game AI with dynamic scripting. *Machine Learning, 63*(3), 217–248. https://doi.org/10.1007/s10994-006-6205-6

Sutoyo, R., Winata, D., Oliviani, K., & Supriyadi, D. M. (2015). Dynamic difficulty adjustment in tower defence. *Procedia Computer Science, 59*, 435–444. https://doi.org/10.1016/j.procs.2015.07.563

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.

Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika, 25*(3–4), 285–294. https://doi.org/10.1093/biomet/25.3-4.285

Unity Technologies. (2026). *Unity 6 user manual*. Retrieved October 28, 2026, from https://docs.unity3d.com/Manual/

Walker, E., & Nowacki, A. S. (2011). Understanding equivalence and noninferiority testing. *Journal of General Internal Medicine, 26*(2), 192–196. https://doi.org/10.1007/s11606-010-1513-8

Watson, G. S. (1964). Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A, 26*(4), 359–372.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin, 1*(6), 80–83. https://doi.org/10.2307/3001968

Yannakakis, G. N., & Togelius, J. (2018). *Artificial intelligence and games*. Springer. https://doi.org/10.1007/978-3-319-63519-4

Appendix A. Glossary of Technical Terms

The words below are used in this proposal. They are listed in alphabetical order, with a short explanation in simple words.

Table 15: Glossary of technical terms

Term	Meaning in simple words
A-star (A*)	A common method for finding the shortest path for one character at a time.
Adaptive	Changes its behaviour depending on what the player does.
AI director	The part of a game that decides what the player faces next, for example which enemies to send and when.
Attack type (strategy)	One kind of wave the director can send, such as sappers at the thinnest wall or flyers over the maze.
Bandit (multi-armed bandit)	A learning problem named after slot machines: repeatedly choose one of several options, see how well it went, and learn which option is best.
Baseline (normal waves)	The version of the game without the director, where waves only get stronger in a fixed way. It is what the director is compared against.
Bernoulli draw	A weighted coin flip: with a reward of 0.7 it gives “success” 70% of the time.
Beta distribution	A way to describe how likely each success rate is, using two counts: α (about the successes) and β (about the failures). More results make it more certain.
Bin (group)	A range that situations are sorted into, for example “very short maze” to “very long maze”. The learner keeps separate results for each group.
Brier score	A number that shows how far predicted chances are from what really happened; lower is better.
Calibration	Whether predicted chances are realistic: if the learner says 70%, it should be right about 70% of the time.
Checkpoint (gate)	A test with pass and fail numbers fixed in advance (G1–G7), and a planned action if it fails.
Collector	A building on an ore deposit that gives the player ore after each wave.
Contextual bandit	A bandit that first looks at the situation (here, the player’s base) before choosing, so it can learn which option is best in which situation.
Core	The building the player must protect; the game is lost when it has no health left.
Counterbalancing	Changing the order of the two versions between players, so that “played second” does not favour one version.
Deterministic (repeatable)	Always giving exactly the same result for the same input.
Dynamic difficulty adjustment (DDA)	Making a game harder or easier while it is played, based on how well the player is doing.
Effect size (dz)	How big a difference is, not just whether it exists. dz is the version for comparing two results from the same person.
Entropy (Shannon entropy)	A number that measures how spread out something is. Here: 0 means all building was in one area, 1 means it was spread evenly over the map.
Expected calibration error (ECE)	The average gap between predicted chances and real results, used to check calibration.
Flow field	A map in which every tile stores the direction to step next to reach the target, so many enemies can find their way cheaply.

Term	Meaning in simple words
Flyer	An enemy that flies in a straight line over walls and rocks; only some towers can hit it.
Hash (fingerprint)	A short number calculated from a large amount of data. If anything in the data changes, the number changes, so two game states can be compared quickly.
Heuristic (hand-written score)	A rule written by the designer that gives each option a score, without learning.
Kernel estimate (Nadaraya–Watson)	An average of past results where results from more similar situations count more.
LinUCB	A well-known bandit method that assumes the reward changes evenly (in a straight line) with the situation.
LLM (large language model)	An AI that generates text, such as ChatGPT. Not used in this project.
Mono	One of Unity's ways of running C# code; the study build uses it because the repeatable core was tested on it.
Non-inferiority test	A test that checks that something is "not worse by more than a set amount", instead of checking that it is better.
Ore	The game's only resource, used to buy everything.
p-value ($p < .05$)	The chance of seeing a result at least this strong if there were really no difference. Below 0.05 is usually taken as a real difference.
PCG32	A fast random number generator that gives the same sequence every time it starts from the same seed.
Pilot (trial study)	A small study (here 5 players) run before the main study to find problems early.
Power (in the game)	A tower only shoots when it is close enough to the core, a collector or a pylon, which each power a limited number of towers.
Pylon	A building that does not shoot but gives power to nearby towers.
Raider	An enemy that attacks the player's ore collectors instead of the core.
Regret bound	A proof of how much worse a learning method can do compared with always choosing the best option.
Replay	A saved file with the seed and every player command, which plays the same game again exactly.
Reward	The score the learner gets after a choice; here it is mostly about whether the player changed the base.
Sapper	An enemy that digs through the player's walls much more cheaply than other enemies.
Seed	A starting number for the random number generator; the same seed always gives the same map and the same random events.
Siege enemy	A slow, tough enemy that stops out of range of most towers and shoots them, switching them off until the next wave.
SPFA	A method for finding the cheapest path to every tile by spreading outward from the target, like a flood, and improving costs until nothing changes.
Telemetry	Data the game saves about each wave, for analysis later.
Threat budget (enemy price list)	The total "price" of the enemies in a wave, the same for every attack type at the same wave number, so all waves are equally strong.
Thompson sampling	A way of choosing: draw a random guess of how good each option is from what has been learned, and pick the best guess. Unsure options get tried sometimes; good ones most of the time.

Term	Meaning in simple words
Threshold	A point where a result changes suddenly, for example the maze length at which sending sappers starts to pay off.
Tick	One fixed step of the game; here 0.02 seconds of game time.
Unity	The game engine used to show the game and read the mouse and keyboard.
Wilcoxon signed-rank test	A statistics test that compares two results from the same people (here, two sessions per player) without assuming the data follows a bell curve.
Within-subject design	A study design in which every player tries both versions, so each player is compared with themselves.